SAGEVM

Technical Reference

Version	v0.9.7
Organization	Night-Traders-Dev
Primary Language	SageLang / C
Document Type	Internal Technical Reference
Status	Active Development

Pure-SageLang implementation of the Sage General Virtual Machine (SGVM) toolchain. Comprising a bytecode compiler and interpreter, SageVM enables self-hosted, portable Sage bytecode execution across all platforms where the Sage interpreter is available. Serves as both the reference implementation and bootstrap tool for SageLang's broader platform support strategy.

SageVM is a pure-SageLang implementation of the Sage General Virtual Machine (SGVM) toolchain, comprising a bytecode compiler (**sgvmc**) and interpreter (**sgvm**). It represents a critical piece of the SageLang ecosystem: a self-hosted, portable VM that enables Sage bytecode to run in environments where only the Sage interpreter is available, serving as both a reference implementation and a bootstrap tool for the language's broader platform support.

1. Architecture & Design Philosophy

1.1 Position in the SageLang Stack

SageVM sits at a specific layer in SageLang's multi-backend architecture, bridging the C-based frontend compiler and the pure-Sage runtime:

Layer	Component	Implementation
Source	.sage files	User code
Frontend	Sage compiler (C)	Lexer, Parser, AST
Intermediate	.svm output	Human-readable VM assembly
Tooling	SageVM (sgvmc / sgvm)	Pure SageLang implementation
Binary	.sgvm files	Serialized bytecode binaries
Execution	MetalVM / SGVM	C-based VM or pure Sage interpreter

The repository's stated purpose is to allow "compilation and execution of SageLang bytecode in a pure SageLang environment" — meaning you can compile and run Sage bytecode using only Sage itself, without requiring the C runtime or MetalVM.

1.2 Core Components

The repository contains three primary source modules:

1. `sgvm_core.sage` — Opcode definitions and shared utilities (`SGVMUtils` class)
2. `sgvm_compiler.sage` — The bytecode compiler/linker (`SGVMCompiler` class)
3. `sgvm.sage` — The interpreter/executor (stack-based VM)

Plus diagnostic variants (`sgvm_compiler_debug.sage`) and unified CLI tooling.

2. The Bytecode Format (.sgvm)

2.1 Binary Structure

The `.sgvm` format is a custom binary serialization. Header structure (from `sgvm_compiler.sage`):

```
[Shebang line]      (optional, e.g. #!/usr/bin/env sgvm\n)
"SGVM"              (4-byte magic)
0x01 0x00           (version major/minor)
function_count      (2 bytes, big-endian)
constant_count      (2 bytes, big-endian)
[Constants Pool]    (variable length)
[Chunk Table]       (4-byte count, then chunks)
```

2.2 Constant Pool

The constant pool supports two entry types:

- **Numbers** (`type=1`): IEEE 754 double-precision, manually encoded via bit manipulation
- **Strings** (`type=3`): Length-prefixed (2-byte BE) UTF-8 data

The compiler deduplicates constants using a `const_map` dictionary keyed by `"n"` + stringified number or `"s"` + string content.

2.3 Instruction Encoding

Instructions use a stack-machine architecture with variable-length operands:

Operand Size	Usage
1 byte	Small immediates (e.g., OP_CALL arg count, OP_DUP depth)
2 bytes (BE16)	Constant indices, local indices, jump offsets
4 bytes (BE32)	Chunk code lengths, chunk table entries

3. The Compiler (sgvm_compiler.sage)

3.1 Two-Pass Architecture

The compiler implements a classic two-pass assembler:

Pass 1 — Symbol Resolution

- Parses `.svm` intermediate format (generated by `sage --emit-vm`)
- Counts functions and chunks; builds local-to-global constant mapping tables
- Handles parameter name resolution (mapping parameter names to `__argN` slots)

Pass 2 — Code Generation

- Emits binary header and serializes constant pool
- Translates opcodes and remaps local indices to global constant pool indices
- Emits chunk metadata and bytecode with proper big-endian encoding

3.2 Opcode Remapping

A critical function of the compiler is opcode translation between the host compiler's bytecode numbering and the VM's expected numbering. The `second_pass` contains explicit mappings:

```
if op == 0x34: op = 52  # BC_OP_IMPORT
elif op == 0x26: op = 38 # BC_OP_CALL_METHOD
elif op == 0x25: op = 37 # BC_OP_CALL
# ... additional remappings ...
```

> This indicates the `.svm` format uses a different opcode numbering scheme than the runtime VM — SageVM acts as a binary translator bridging these two numbering worlds.

3.3 Security Considerations

The compiler includes command injection protection when shelling out to `sage --emit-vm`:

- Validates input/output paths against forbidden shell metacharacters (`;`, `&`, `|`, `$`, etc.)
- Explicitly allows spaces but rejects quotes and redirection operators

3a. How `sage --emit-vm` Works

The `sage --emit-vm` command is the SageLang compiler's bytecode emission mode. It takes a `.sage` source file and produces a human-readable intermediate representation (`.svm`) rather than a binary. This `.svm` file is then consumed by `sgvmc` to produce the final `.sgvm` binary.

Compilation Pipeline

```
.sage source  ■■■  sage --emit-vm  ■■■  .svm file  ■■■  sgvmc  ■■■  .sgvm binary
     ■                                      ■                ■
  Frontend                              Intermediate      Linker /
 (C compiler)                           (text format)    Serializer
```

The `.svm` format serves as a stable textual ABI between the C-based Sage compiler and the pure-SageLang VM toolchain. This decoupling means:

- The C compiler doesn't need to know the `.sgvm` binary format
- The SageVM compiler doesn't need to parse Sage source code directly
- Changes to either side only require updating the `.svm` parser, not both

The .svm Output Structure

The `.svm` format is a line-oriented text format with explicit section markers:

1 — Top-Level Metadata

```
functions <count>
chunks <count>
```

Declares the total number of function definitions and code chunks in the file.

2 — Chunk / Function Declaration

```
chunk       # top-level script code (main program body)
function    # a named function definition
```

3 — Parameter Section (Functions Only)

```
params <count>
<param_len>
<hex_encoded_param_name>     # repeated <count> times

# Example: function("name", "age")
function
params 2
4
6E616D65        # "name"
3
616765          # "age"
```

4 — Constants Section

```
constants <count>

# Number constant:
number 3.14159

# String constant:
string 5
48656C6C6F      # "Hello"
```

5 — Code Section

```
code <byte_count>
<hex_encoded_bytecode>     # two hex digits per byte

# Example: code 5 followed by 0102030405
# Represents bytes: 0x01 0x02 0x03 0x04 0x05
```

Opcode Remapping Table

Opcodes not present in this table pass through the compiler unchanged:

Host Opcode	Host Name	VM Opcode	VM Name
0x08	BC_OP_DEFINE_FUNCTION	8	OP_DEFINE_FUNCTION
0x09	BC_OP_GET_PROPERTY	9	OP_GET_PROPERTY
0x0a	BC_OP_SET_PROPERTY	10	OP_SET_PROPERTY
0x0b	BC_OP_GET_INDEX	11	OP_GET_INDEX
0x0c	BC_OP_SET_INDEX	12	OP_SET_INDEX
0x0d	BC_OP_LOAD_FUNCTION	13	OP_LOAD_FUNCTION
0x0e	BC_OP_SLICE	14	OP_SLICE
0x25	BC_OP_CALL	37	OP_CALL
0x26	BC_OP_CALL_METHOD	38	OP_CALL_METHOD
0x33	BC_OP_LOOP_BACK	51	OP_LOOP_BACK
0x34	BC_OP_IMPORT	52	OP_IMPORT
0x35	BC_OP_CLASS	53	OP_CLASS
0x36	BC_OP_METHOD	54	OP_METHOD
0x37	BC_OP_INHERIT	55	OP_INHERIT
0x38	BC_OP_SETUP_TRY	56	OP_SETUP_TRY
0x39	BC_OP_END_TRY	57	OP_END_TRY
0x44	BC_OP_RAISE	58	OP_RAISE

Design Rationale

The `.svm` intermediate format serves several architectural purposes:

1. **Decoupling:** The C compiler and pure-Sage VM evolve independently
2. **Debugging:** Human-readable text makes it trivial to inspect compiler output
3. **Verification:** The `diff_bytecode.sage` tool can compare `.svm` traces between interpreted and compiled compiler runs
4. **Portability:** The `.svm` parser is simple enough to reimplement in any language
5. **Self-Hosting Path:** Eventually a SageLang-written compiler could emit `.svm` directly, completing the bootstrap

The hex encoding of strings and bytecode ensures exact byte-level reproducibility and eliminates ambiguity about string encoding or byte values — critical when the same format must be parsed identically by C and SageLang.

4. The Interpreter (sgvm.sage)

4.1 Execution Model

- **Stack-based execution:** All operations work on an operand stack
- **Environment frames:** OP_PUSH_ENV / OP_POP_ENV for scope management
- **Chunk-based organization:** Functions and top-level code are separate chunks
- **Constant pool access:** Global constants referenced by 16-bit indices

4.2 Opcode Set (89 Opcodes)

The VM supports 89 opcodes (0–87, plus 255 for HALT), categorized as follows:

Category	Key Opcodes	Count
Stack Operations	CONSTANT, NIL, TRUE, FALSE, POP, DUP	6
Variable Access	GET_GLOBAL, DEFINE_GLOBAL, SET_GLOBAL, GET/SET_PROPERTY, GET/SET_INDEX	7
Arithmetic	ADD, SUB, MUL, DIV, MOD, NEGATE	6
Comparison	EQUAL, NOT_EQUAL, GREATER, GREATER_EQUAL, LESS, LESS_EQUAL	6
Bitwise	BIT_AND, BIT_OR, BIT_XOR, BIT_NOT, SHIFT_LEFT, SHIFT_RIGHT	6
Logic	NOT, TRUTHY	2
Control Flow	JUMP, JUMP_IF_FALSE, LOOP_BACK, BREAK, CONTINUE	5
Functions	DEFINE_FUNCTION, LOAD_FUNCTION, CALL, CALL_METHOD, RETURN	5
Data Structures	ARRAY, TUPLE, DICT, ARRAY_LEN, SLICE	5
OOP	CLASS, METHOD, INHERIT	3
Exceptions	SETUP_TRY, END_TRY, RAISE	3
Modules	IMPORT, EXEC_AST_STMT	2
Environment	PUSH_ENV, POP_ENV	2
I/O	PRINT	1
GPU Hot-Path	GPU_POLL_EVENTS through GPU_CMD_DISPATCH (opcodes 59–86)	28
Math	MATH_PRINTM	1
System	HALT (0xFF)	1

4.3 GPU Acceleration Opcodes

Notably, 28 opcodes (59–86) are dedicated to GPU hot-path operations, enabling real-time graphics without leaving the VM:

- **Window/input handling:** POLL_EVENTS, KEY_PRESSED, MOUSE_POS
- **Command buffer building:** BEGIN_COMMANDS, END_COMMANDS

- **Rendering operations:** `CMD_DRAW`, `CMD_BIND_VB`, `CMD_DISPATCH`
- **Synchronization:** `SUBMIT_SYNC`, `WAIT_FENCE`

> This is a significant architectural decision — the VM provides first-class GPU abstractions, not just CPU abstraction. SageVM targets game engines, simulations, and graphical applications running in pure-Sage environments.

5. The sgvm_core.sage Utilities

The `SGVMUtils` class provides low-level binary manipulation. Its manual implementations reveal important design priorities:

5.1 Manual IEEE 754 Double Encoding

Custom double-precision floating-point serialization without built-in binary packing:

- Manual extraction of sign, exponent, and 52-bit mantissa
- Bit-by-bit construction of 8 output bytes
- Special-case handling for zero (`0.0`) and nil

5.2 Hex Parsing

Custom hex-to-byte conversion for reading the `.svm` intermediate format:

- Case-insensitive (`A-F` normalized to `a-f`)
- Character-by-character lookup against `"0123456789abcdef"`

5.3 String Handling

- Custom `split_lines` supporting both `\\n` and `\\r\\n`
- Custom `trim` using ASCII codepoint comparison (`ord(c) <= 32`)
- Custom `my_substr` for bounded string slicing

These utility implementations prioritize explicit control over standard library dependencies — likely for self-hosting reliability and educational transparency.

6. Tooling & Developer Experience

6.1 Unified CLI (sagevm)

The project uses a modern unified binary with subcommands:

```
sagevm run <file.sgvm>      # Execute bytecode
sagevm compile <file.sage>  # Compile to binary
sagevm dis <file.sgvm>      # Disassemble
sagevm hex <file.sgvm>      # Hexdump
sagevm version              # Version info
```

Legacy symlinks maintain backward compatibility:

- `sgvm` → `sagevm run`
- `sgvmc` → `sagevm compile`

6.2 Diagnostic Tools

Tool	Purpose
`diff_bytecode.sage`	Compare execution traces or hex diffs between interpreted vs compiled compiler runs

sgvm_disassembler.sage	Convert .sgvm binaries back to readable Sage source representation
sgvm_compiler_debug.sage	Instrumented compiler emitting DIAG traces for every opcode — Phase 2 debugging

The diff tool compares `j_after` (stream position after opcode), `global_idx` (constant pool resolution), and `op` (opcode numbers). Its existence indicates the project has reached a maturity level requiring formal verification of compiler correctness.

7. Performance Characteristics

7.1 Benchmarks (v0.9.7)

Benchmark	Duration (ms)	Workload Type
01_fibonacci.sage (fib(22))	2,256	Recursive computation
02_loop_sum.sage	4,513	Integer accumulation
03_string_concat.sage	495	String operations
04_array_ops.sage	2,644	Array manipulation
05_dict_ops.sage	2,542	Hash map operations
06_class_method.sage	7,876	OOP dispatch overhead
07_nested_loops.sage	16,991	Deep loop nesting
08_exception_handling.sage	417	Try / catch / finally
10_primes_sieve.sage	1,131	Algorithmic (Sieve of Eratosthenes)

Key Observations:

- **Exception handling is fast (417 ms)** — suggests efficient stack unwinding implementation
- **Class methods show significant overhead (7,876 ms)** — OOP dispatch in a pure interpreter is expensive; each method call traverses the environment chain
- **Nested loops are the slowest (16,991 ms)** — likely due to environment frame allocation/deallocation overhead on each iteration
- **String concatenation is surprisingly fast (495 ms)** — possibly due to immutable string optimizations or special-casing in the runtime

7.2 Performance Context

These numbers represent pure-interpreter overhead — the VM is implemented in SageLang, which itself runs on the C-based Sage interpreter. This is effectively a meta-circular interpreter (interpreter interpreting bytecode), so performance is expected to be orders of magnitude slower than native execution. The numbers are expected for this architecture and do not reflect a fundamental design flaw.

8. Integration & Ecosystem Role

8.1 Submodule Architecture

SageVM is designed as a submodule at `core/src/sage/vm-tools` within the main SageLang repository. This positioning enables three strategic scenarios:

1. **Bootstrap path:** The C-based `sage` compiler generates `.svm` files; SageVM compiles them to `.sgvm` binaries — no additional C tooling required
2. **Self-hosting bridge:** The self-hosted Sage interpreter (written in Sage) can use SageVM to execute bytecode entirely without C dependencies

3. **Cross-platform portability:** Pure Sage code runs anywhere Sage runs, making `.sgvm` files executable on platforms where C compilation is unavailable

8.2 Specification Lockstep

> The documentation mandates that opcodes remain in "100% lockstep" with `core/src/vm/bytecode.h` in the main repository. This is critical — the pure-Sage VM must behave identically to the C-based MetalVM to ensure ecosystem consistency and portability guarantees.

9. Strengths & Technical Merits

1. **Educational Clarity:** Explicit, manual implementations (IEEE 754 encoding, hex parsing) make every internal mechanism transparent to students and contributors
2. **Self-Hosting Completeness:** Enables a full bootstrap path from C compiler → Sage self-host → pure-Sage VM; a rare achievement for a language at this scale
3. **GPU-First Design:** 28 dedicated GPU opcodes demonstrate forward-thinking architecture for graphics and simulation workloads — unusual for a VM at this maturity level
4. **Diagnostic Rigor:** The diff tool and DIAG trace system constitute production-grade debugging infrastructure, not just development scaffolding
5. **Security Consciousness:** Path validation against command injection reflects a security-aware implementation posture
6. **Unified UX:** Modern CLI design (`sagevm` with subcommands) with backward-compatible legacy symlinks for ecosystem continuity

10. Weaknesses & Areas for Improvement

1. **Performance:** Meta-circular interpreter architecture means performance is inherently limited — no JIT compilation within the pure-Sage layer
2. **Memory Overhead:** Stack machine with environment frames on each scope boundary likely causes significant GC pressure in the host interpreter
3. **Opcode Hardcoding:** Explicit remapping tables in `second_pass` are brittle — any change to `bytecode.h` requires a coordinated manual update
4. **Error Handling:** The compiler returns `false` on errors with no structured error reporting system; absence of source line numbers makes debugging difficult
5. **Limited Optimizations:** No constant folding, dead code elimination, or peephole optimization at the VM level
6. **String Handling:** Custom string manipulation primitives instead of built-ins suggest either compatibility constraints or unnecessary divergence from idiomatic SageLang

11. Comparative Context

Feature	SageVM	Python VM	Lua VM	JVM
Implementation	Pure SageLang	C	C	C / C++
Self-Hosted	Yes (partial)	No	No	No
GPU Opcodes	Yes (28)	No	No	No (JNI only)
Stack Model	Stack machine	Stack machine	Register + Stack	Stack machine
Binary Format	Custom (.sgvm)	.pyc (marshal)	.luac	.class
Exception Support	Native try / catch	Native	Native (pcall)	Native
OOP	Classes + Inheritance	Classes	Prototypes	Classes

SageVM occupies a unique niche: more capable than a toy VM (GPU support, full OOP, exceptions) but intentionally simpler than industrial VMs by virtue of being self-hosted and educational. It demonstrates that a meaningful VM can be built in a high-level, indentation-based language without sacrificing architectural coherence.

12. Conclusion

SageVM v0.9.7 is a sophisticated, well-architected educational and tooling VM that serves as a critical bridge in the SageLang ecosystem. Its pure-SageLang implementation enables bootstrap scenarios and cross-platform portability that would be impossible with C-only tooling. The inclusion of GPU opcodes, comprehensive OOP support, and rigorous diagnostic tools demonstrate a project that has evolved beyond a simple learning exercise into a genuine piece of language infrastructure.

The codebase prioritizes clarity, correctness, and ecosystem integration over raw performance — an appropriate tradeoff for its role as a reference implementation and bootstrap tool.

Future development would benefit most from:

- A register-based VM redesign to reduce environment frame allocation overhead
- A structured error reporting system with source line number propagation
- Automated opcode synchronization tooling to eliminate manual remapping table maintenance

For developers interested in VM construction, SageVM provides an excellent case study in how to build a complete, self-hosted bytecode toolchain — with modern language features including exceptions, classes, and GPU compute — in a high-level, indentation-based language.

End of Document · SageVM Technical Reference v0.9.7 · Night-Traders-Dev